Grupo – VetCare

VetCare

Documento de Arquitetura de Software

Versão 4.0

Histórico da Revisão

Data	Versão	Descrição	Autor
29/mai/2026	1.0	Criação inicial do Documento de Arquitetura de Software	Grupo
18/jun/2026	2.0	Atualização do escopo e da arquitetura para o VetCare	Grupo
25/jun/2026	3.0	Correção da arquitetura lógica e segurança de dados	Níchollas
27/jun/2026	3.1	Correção sobre coerência na arquitetura	Grupo
28/jun/2026	4.0	Revisão das seções arquiteturais, atualização dos diagramas e alinhamento com os requisitos definidos	Grupo

Índice Analítico

1. Introdução **4**
1.1 Finalidade 4
1.2 Escopo 4
1.3 Definições, Acrônimos e Abreviações 5
2. Restrições e requisitos arquiteturais **7**
3. Visão de Cenários **8**
3.1 Lista de Histórias de Usuário 8
4. Visão Lógica **9**
4.2 Decisões arquiteturais 11
4.3 Representação da arquitetura lógica 11
4.4 Representação do funcionamento da arquitetura 13
5. Visão de Implantação **15**
6. Visão de Dados **16**
7. Volume e Desempenho **18**
7.1 Estimativa de Usuários e Acessos 18
7.2 Dimensionamento de Dados 18
7.3 Metas de Desempenho e Disponibilidade 18
8. Referências **18**

Documento de Arquitetura de Software

1. Introdução

A introdução deste Documento de Arquitetura de Software apresenta uma visão geral da organização arquitetural do sistema VetCare e orienta a leitura das demais seções. O documento descreve a finalidade da arquitetura, o escopo da solução, os principais termos utilizados, os requisitos arquiteturais, a visão de cenários, a visão lógica, a visão de implantação, a visão de dados, as características de volume e desempenho e as referências utilizadas.

1.1 Finalidade

Este documento tem como finalidade apresentar a arquitetura de software do sistema VetCare, registrando as principais decisões arquiteturais adotadas para sua construção e evolução.

O objetivo é comunicar, de forma organizada, como o sistema está estruturado, quais tecnologias foram selecionadas, quais atributos de qualidade foram considerados e como os componentes da aplicação se relacionam para atender aos requisitos definidos no Documento de Requisitos de Software.

A organização deste documento é inspirada no modelo 4+1 de Kruchten (1994), utilizado para representar a arquitetura a partir de diferentes perspectivas. No contexto do VetCare, esse modelo é usado como estrutura de documentação arquitetural, contemplando a visão de cenários, a visão lógica, a visão de implantação e a visão de dados.

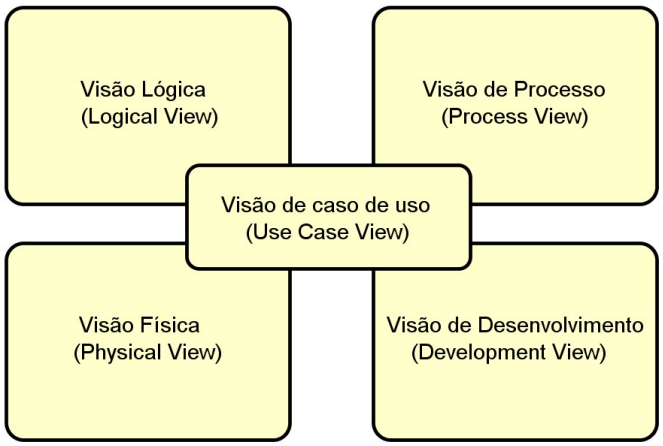


Figura 1 – Visões 4+1 da arquitetura de software

Como decisão arquitetural interna da aplicação, o VetCare adota uma **arquitetura em camadas**, associada ao padrão **MVC** — Model, View, Controller. Dessa forma, o **modelo 4+1** organiza a documentação, a arquitetura em camadas organiza a solução lógica do sistema, e o padrão MVC apoia a separação de responsabilidades entre apresentação, controle das requisições e representação dos dados da aplicação.

Este documento é destinado aos integrantes da equipe de desenvolvimento, análise, qualidade e gestão do projeto, servindo como referência para implementação, manutenção, validação e evolução futura do sistema.

1.2 Escopo

O VetCare é uma plataforma web de gestão clínica veterinária, desenvolvida inicialmente para a Clínica VetSPA, especializada em fisioterapia veterinária.

O sistema tem como objetivo substituir processos manuais e fragmentados, como cadernos, planilhas avulsas e aplicativos genéricos de mensagem, por uma solução centralizada para controle de pacientes, tutores, veterinários, agenda, prontuários, registros clínicos, documentos, mídias e comunicação.

Embora tenha origem nas necessidades específicas da VetSPA, a arquitetura do VetCare foi pensada para permitir evolução futura como solução **SaaS/Multi-Tenant**, possibilitando que diferentes clínicas veterinárias utilizem a mesma plataforma com separação lógica de dados, usuários, permissões e configurações por clínica.

O sistema contempla quatro perfis principais de uso: **Administrador**, **Veterinário**, **Tutor** e **Apoio administrativo**. O **Administrador** é responsável pela gestão geral da clínica e dos dados consolidados; o **Veterinário** realiza avaliações, atendimentos e acompanhamento clínico; o **Tutor** acessa informações autorizadas sobre o tratamento do pet; e o **Apoio administrativo** atua como um perfil operacional com permissões limitadas, auxiliando em cadastros, agenda e confirmação de atendimentos, sem possuir acesso completo às configurações administrativas.

O escopo funcional do VetCare inclui autenticação de usuários por perfil, gerenciamento de usuários, tutores e pacientes, agenda individual do veterinário, agenda geral da clínica, confirmação ou cancelamento de presença em sessão, registro de avaliação clínica, registro de atendimento fisioterapêutico, observações internas, anexos de fotos e vídeos, gerenciamento de documentos clínicos, consulta ao prontuário, comunicação entre tutor e veterinário, notificações e dashboards operacionais.

Não fazem parte do escopo inicial funcionalidades como emissão fiscal, controle financeiro avançado, integração com meios de pagamento, documentação sanitária para viagens, compartilhamento universal de prontuário por QR Code ou uso de inteligência artificial para diagnóstico clínico.

1.3 Definições, Acrônimos e Abreviações

Termo	*Definição*
VetCare	Sistema web de gestão clínica veterinária, com foco inicial em fisioterapia veterinária.
VetSPA	Clínica veterinária cliente que originou a necessidade do sistema.
SaaS	Software como Serviço; modelo em que a aplicação é disponibilizada pela internet.
Multi-Tenant	Modelo arquitetural no qual uma mesma aplicação atende diferentes organizações, mantendo separação lógica dos dados, usuários, permissões e configurações.
Tenant	Unidade cliente dentro da plataforma, como uma clínica cadastrada no sistema.
Domínio	Conjunto de conceitos, entidades, regras e relações que representam o negócio atendido pelo sistema. No VetCare, envolve elementos como clínica, usuários, tutores, pacientes, tratamentos, sessões, prontuários e atendimentos.
MVC	Model, View, Controller; padrão arquitetural utilizado para separar responsabilidades entre apresentação, controle das requisições e representação dos dados da aplicação.
Model	Componente responsável por representar os dados e entidades do domínio da aplicação, servindo de base para as regras de negócio e para a persistência.
View	Componente responsável pela apresentação das telas e pela interação visual com o usuário.
Controller	Componente responsável por receber requisições, validar entradas, coordenar o fluxo da aplicação e acionar os serviços de negócio adequados.

Administrador	Usuário responsável pela gestão da clínica, usuários, pacientes, permissões, agenda geral e dados consolidados.
Veterinário	Profissional que realiza avaliações clínicas, sessões de fisioterapia, registros de atendimento e acompanhamento da evolução dos pacientes.
Tutor	Responsável legal pelo pet em tratamento, com acesso às informações autorizadas do prontuário e da agenda do animal.
Apoio administrativo	Equipe de recepção ou suporte operacional da clínica, responsável por auxiliar no cadastro, organização da agenda e confirmação de atendimentos.
Paciente/Pet	Animal submetido ao tratamento fisioterapêutico.
Tratamento	Processo terapêutico associado a um paciente/pet, podendo agrupar avaliação clínica, sessões fisioterapêuticas, atendimentos, registros de evolução, mídias e documentos clínicos relacionados.
Prontuário	Conjunto cronológico de registros clínicos do paciente, incluindo avaliações, atendimentos, mídias, documentos e evolução clínica.
Sessão	Agendamento ou ocorrência de atendimento fisioterapêutico realizado pelo veterinário.
Atendimento fisioterapêutico	Registro clínico realizado durante ou após uma sessão, contendo técnicas aplicadas, escala de dor, evolução clínica, sinais vitais e próximos passos.
Observação interna	Anotação clínica restrita aos perfis Administrador e Veterinário, não visível ao Tutor.
HU	História de Usuário.
RN	Regra de Negócio.
RNF	Requisito Não Funcional.
API	Interface de Programação de Aplicações, utilizada para comunicação entre sistemas ou camadas da aplicação.
REST	Estilo arquitetural baseado em recursos HTTP, utilizado para estruturar a comunicação entre front-end e back-end.
JWT	Token utilizado para autenticação e autorização de usuários no sistema.
Auto-Save	Salvamento automático de dados durante o preenchimento de formulários clínicos, reduzindo risco de perda de informações.

Tabela 1 – definições, acrônimos e abreviações

2. Restrições e requisitos arquiteturais

Atributo de qualidade	*Requisito de arquitetura*	*Solução*
Desempenho	Consultas de agenda e prontuário devem responder em até 2 segundos para 95% das requisições, conforme RNF-004.	Utilização de consultas otimizadas, paginação de registros, índices em campos de busca, carregamento sob demanda de mídias e separação entre dados clínicos e arquivos anexos.
Interoperabilidade	O sistema deve permitir futura integração com serviços externos e importação de dados estruturados.	Adoção de API REST para comunicação entre front-end e back-end, além de suporte futuro à importação de arquivos CSV/XLSX e integração com serviços externos.
Usabilidade	A interface deve ser simples, responsiva e adequada ao uso clínico rápido por administradores, veterinários, tutores e apoio administrativo.	Interface web responsiva em React, navegação por sidebar fixa, formulários objetivos por perfil e aderência ao design system definido.
Confiabilidade	Registros clínicos confirmados não devem ser perdidos ou excluídos definitivamente, mantendo histórico completo do paciente.	Uso de persistência transacional, controle de integridade no banco de dados, rastreabilidade de alterações, soft delete para registros sensíveis e recurso de Auto-Save em formulários clínicos.
Segurança	O sistema deve proteger dados clínicos e restringir acessos conforme o perfil do usuário.	Autenticação por JWT, controle de acesso por perfil, uso de Spring Security, senhas armazenadas com hash, expiração de sessão e filtragem de dados sensíveis no back-end.
Facilidade de manutenção	O sistema deve permitir evolução dos módulos sem comprometer toda a aplicação.	Adoção de arquitetura em camadas associada ao padrão MVC, separando apresentação, controle, serviços de negócio, persistência e banco de dados.
Portabilidade	O sistema deve funcionar em diferentes ambientes, navegadores e dispositivos.	Aplicação web responsiva, compatibilidade com navegadores modernos e empacotamento em containers Docker para facilitar implantação em diferentes ambientes.
Escalabilidade	O sistema deve suportar crescimento no número de clínicas, usuários, pacientes, prontuários, documentos e mídias.	Arquitetura preparada para evolução SaaS/Multi-Tenant, separação lógica por clínica/tenant, armazenamento externo de mídias e possibilidade de escalabilidade horizontal do servidor de aplicação.

Disponibilidade	O sistema deve permanecer acessível para apoiar a rotina clínica, com disponibilidade mensal mínima de 99,5%, excluídas janelas de manutenção programada, conforme RNF-005.	Implantação em nuvem com monitoramento contínuo, health check, backup diário, manutenção programada e mecanismos de recuperação para garantir a continuidade do serviço.

Tabela 2 – Restrições e requisitos arquiteturais

3. Visão de Cenários

A partir do escopo definido e das restrições arquiteturais levantadas, esta seção apresenta os cenários funcionais utilizados como referência para validar a arquitetura proposta.

Como o projeto utiliza Histórias de Usuário como técnica de especificação funcional, esta seção apresenta apenas uma síntese das histórias identificadas, com seus respectivos atores principais e prioridades. As descrições completas, critérios de aceite, dependências, regras de negócio e requisitos não funcionais encontram-se no Documento de Requisitos de Software.

A Visão de Cenários permite manter a rastreabilidade entre os requisitos funcionais e as decisões arquiteturais adotadas no projeto, sem duplicar o conteúdo detalhado do documento de requisitos.

3.1 Lista de Histórias de Usuário

ID	*História de Usuário*	*Ator Principal*	*Prioridade*
HU-001	Autenticar usuário	Administrador, Veterinário, Tutor	Deve ter
HU-002	Gerenciar usuários	Administrador	Deve ter
HU-003	Gerenciar pacientes	Administrador	Deve ter
HU-004	Gerenciar agenda individual do veterinário	Veterinário	Deve ter
HU-005	Visualizar agenda geral da clínica	Administrador/Apoio	Deve ter
HU-006	Confirmar ou cancelar presença em sessão	Tutor	Deve ter
HU-007	Registrar avaliação clínica	Veterinário	Deve ter
HU-008	Registrar atendimento fisioterapêutico	Veterinário	Deve ter
HU-009	Registrar observações internas	Veterinário	Deve ter

HU-010	Anexar mídias à sessão	Veterinário	Deve ter
HU-011	Consultar prontuário	Veterinário/Tutor	Deve ter
HU-012	Gerenciar documentos clínicos	Veterinário	Deveria ter
HU-013	Acompanhar tratamento	Tutor	Deve ter
HU-014	Trocar mensagens	Tutor/Veterinário	Deveria ter
HU-015	Receber notificações	Tutor/Veterinário	Deveria ter
HU-016	Consultar indicadores do dia	Administrador/Veterinário	Deveria ter

Tabela 3 – Lista de Histórias de Usuário

4. Visão Lógica

Com base nos cenários funcionais apresentados na seção anterior e nos atributos de qualidade definidos como requisitos arquiteturais, a Visão Lógica descreve como o VetCare é organizado internamente em classes, camadas e componentes de software.

Essa visão tem como objetivo demonstrar como os principais conceitos do domínio clínico veterinário são representados na aplicação e como a estrutura lógica do sistema apoia funcionalidades essenciais, como autenticação, controle de acesso por perfil, gestão de pacientes, agenda, avaliação clínica, atendimento fisioterapêutico, prontuário, documentos, mídias, comunicação e notificações.

No VetCare, a organização lógica segue uma arquitetura em camadas associada ao padrão MVC. A camada de apresentação permite a interação dos usuários com a interface web; a camada de controle recebe e valida as requisições; a camada de negócio concentra as regras funcionais e regras de negócio; a camada de persistência realiza o acesso aos dados; e o banco de dados armazena as informações clínicas e operacionais do sistema.

A modelagem do domínio foi realizada por meio do Diagrama de Classes da UML, que representa as entidades significativas do sistema, seus atributos, operações e relacionamentos. A organização em camadas e pacotes permite separar responsabilidades, reduzir acoplamento, aumentar a coesão dos módulos e facilitar a manutenção e evolução do sistema.

4.1 Representação do domínio da aplicação

No projeto VetCare, o diagrama de classes foi elaborado a partir das funcionalidades descritas no Documento de Visão e no Documento de Requisitos, mantendo alinhamento com as Histórias de Usuário apresentadas na Visão de Cenários. Entre as entidades significativas do domínio destacam-se Usuário, Administrador, Veterinário, Tutor, Apoio Administrativo, Paciente, Tratamento, Agenda, Sessão, Avaliação Clínica, Atendimento Fisioterapêutico, Prontuário, Documento Clínico, Mídia, Mensagem e Notificação.

Essas classes representam os principais elementos do negócio e servem de base para a implementação das entidades persistentes em Java, utilizando JPA/Hibernate. A relação entre Paciente, Tratamento, Sessão, Avaliação Clínica, Atendimento Fisioterapêutico e Prontuário é especialmente relevante, pois sustenta a rastreabilidade da evolução clínica, a integridade dos registros e a consulta cronológica das informações do paciente.

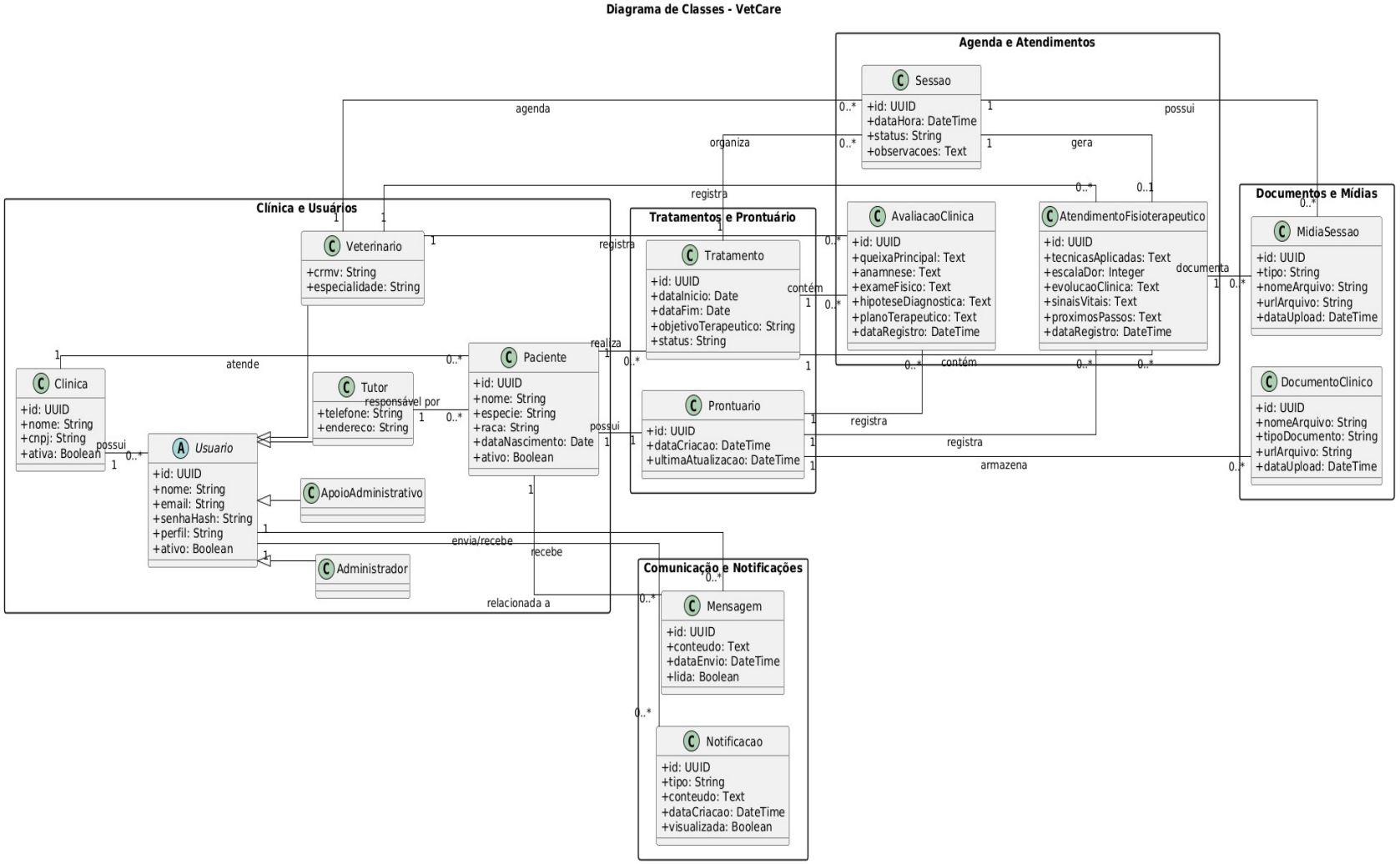


Figura 2 – Diagrama de Classes do VetCare

4.2 Decisões arquiteturais

Item arquitetural	*Decisão*	*Justificativa*
Estilo arquitetural	Arquitetura em Camadas (Layered Architecture) + Arquitetura Cliente-Servidor (3 camadas)	O sistema é uma aplicação web composta por interface, regras de negócio e persistência. Os requisitos não indicam necessidade de microsserviços. A arquitetura em camadas oferece baixo acoplamento, alta manutenibilidade e atende perfeitamente ao MVP, podendo futuramente evoluir para microsserviços caso a clínica cresça.
Linguagem de programação	Java 21 LTS (Back-end) e TypeScript 5.x (Front-end)	Java 21 LTS foi adotado no back-end por oferecer robustez, portabilidade pela JVM, suporte de longo prazo e ampla utilização em sistemas corporativos. TypeScript 5.x foi adotado no front-end por melhorar a manutenção e confiabilidade da interface web por meio da tipagem estática.
Framework adotado	Spring Boot 3.x, Spring Security Spring Data JPA (back) e React 19 (front)	O Spring Boot simplifica o desenvolvimento da API REST, autenticação e persistência dos dados. O React permite construir uma interface web responsiva e moderna, conforme os requisitos do sistema.
Formas de persistência	Banco de dados relacional utilizando JPA/Hibernate e armazenamento externo para mídias	Os dados clínicos possuem forte relacionamento entre si, enquanto fotos, vídeos e documentos devem ser armazenados separadamente para melhor desempenho e escalabilidade.
Banco de dados	PostgreSQL 16	Atende aos requisitos de integridade, confiabilidade e desempenho, oferecendo suporte a transações ACID e relacionamentos complexos entre entidades.
Serviços de terceiros	Serviço de armazenamento de arquivos (Amazon S3 ou MinIO), serviço de e-mail SMTP e WebSocket para comunicação em tempo real	O sistema necessita armazenar mídias, enviar notificações e permitir comunicação em tempo real entre veterinário e tutor, conforme especificado nos requisitos funcionais.

Tabela 4 – Decisões arquiteturais

4.3 Representação da arquitetura lógica

A arquitetura lógica do VetCare é representada por um diagrama de camadas, alinhado ao estilo arquitetural definido na seção de decisões arquiteturais. Essa representação demonstra como as responsabilidades do sistema são distribuídas entre apresentação, controle, serviços de negócio, persistência e banco de dados.

A camada de Apresentação, desenvolvida em React, é responsável pela interface web utilizada pelos perfis Administrador, Veterinário, Tutor e Apoio Administrativo. A camada de Controle, implementada com Spring MVC, recebe as requisições HTTP, valida os dados de entrada e direciona as chamadas aos serviços adequados. A camada de Serviços concentra as regras de negócio, como controle de permissões, validação de agenda, registro clínico,

consulta ao prontuário, envio de notificações e comunicação entre usuários.

A camada de Persistência utiliza Spring Data JPA/Hibernate para abstrair o acesso aos dados e manipular as entidades do domínio por meio de repositórios. O banco de dados PostgreSQL armazena os dados estruturados da aplicação, como usuários, pacientes, sessões, avaliações, atendimentos e prontuários. Fotos, vídeos e documentos clínicos são tratados como arquivos externos, mantendo no banco apenas seus metadados e vínculos com os registros clínicos.

Essa organização permite que alterações na interface, nas regras de negócio ou na persistência sejam realizadas com menor impacto sobre as demais partes do sistema, contribuindo para manutenibilidade, segurança, escalabilidade e evolução futura da aplicação.

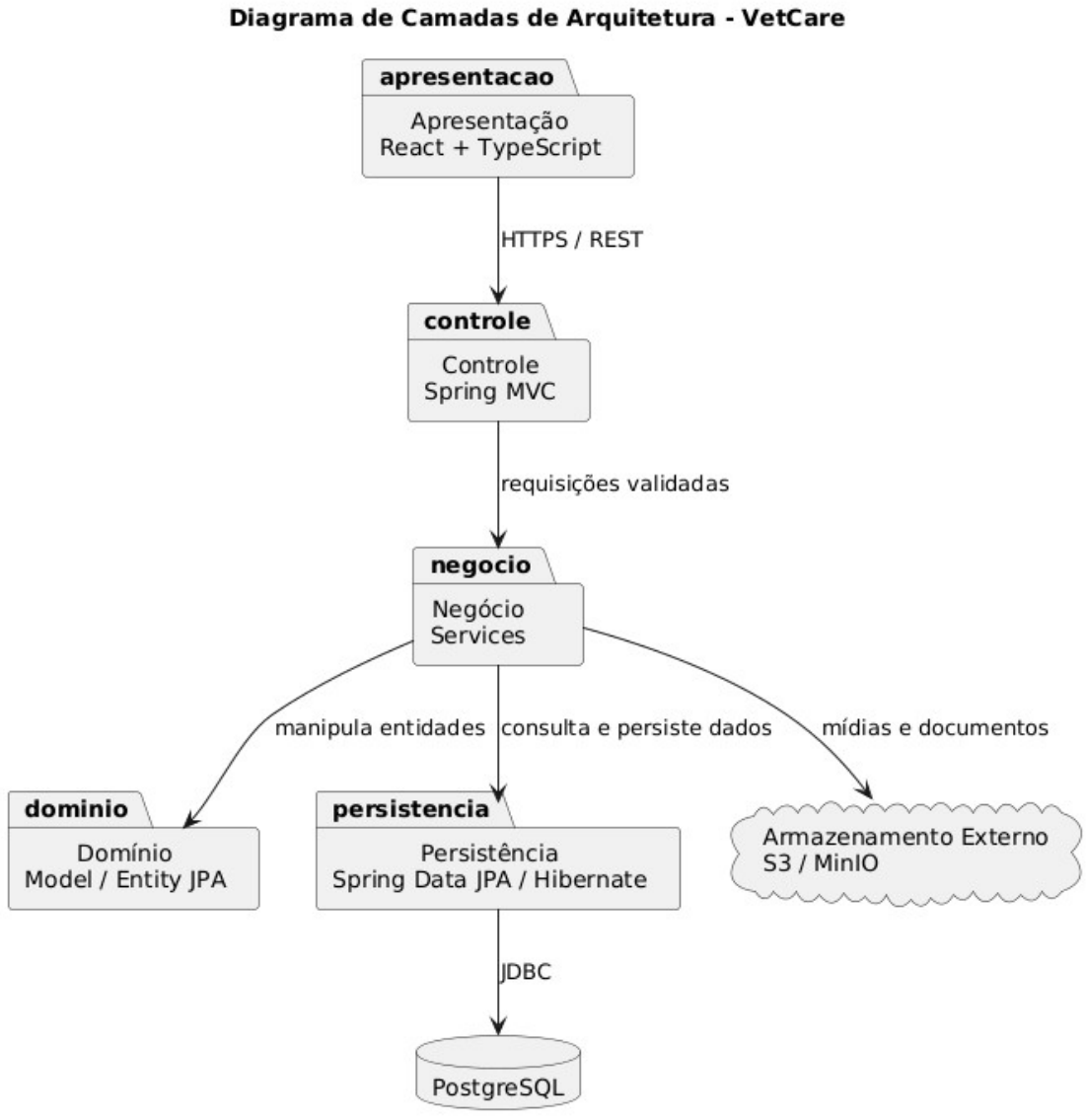


Figura 3 – Diagrama de Camadas de Arquitetura do VetCare

4.4 Representação do funcionamento da arquitetura

O Diagrama de Sequência da UML representa a interação dinâmica entre atores, interfaces, componentes e objetos ao longo do tempo. No VetCare, esse diagrama é utilizado para demonstrar como as camadas da arquitetura colaboram durante a execução de um cenário funcional.

O cenário escolhido para representar o funcionamento da arquitetura é o registro de uma avaliação clínica, pois envolve um fluxo central do sistema e percorre as principais camadas da aplicação. O processo inicia quando o Veterinário preenche os dados da avaliação na interface web. Em seguida, a requisição é enviada ao controller, que valida os dados de entrada e encaminha a solicitação ao service responsável.

Na camada de negócio, o service aplica as regras necessárias, verifica permissões, consulta entidades relacionadas, como Paciente e Prontuário, e cria ou atualiza a entidade Avaliação Clínica. Em seguida, a camada de persistência utiliza os repositórios JPA para gravar as informações no PostgreSQL. Após a confirmação da operação, a resposta retorna ao controller e, posteriormente, à interface, que exibe a confirmação ao usuário.

Esse fluxo demonstra a colaboração entre interface, controller, service, entidades de domínio, repositories e banco de dados, evidenciando como a arquitetura definida sustenta requisitos como segurança, confiabilidade, rastreabilidade e facilidade de manutenção.

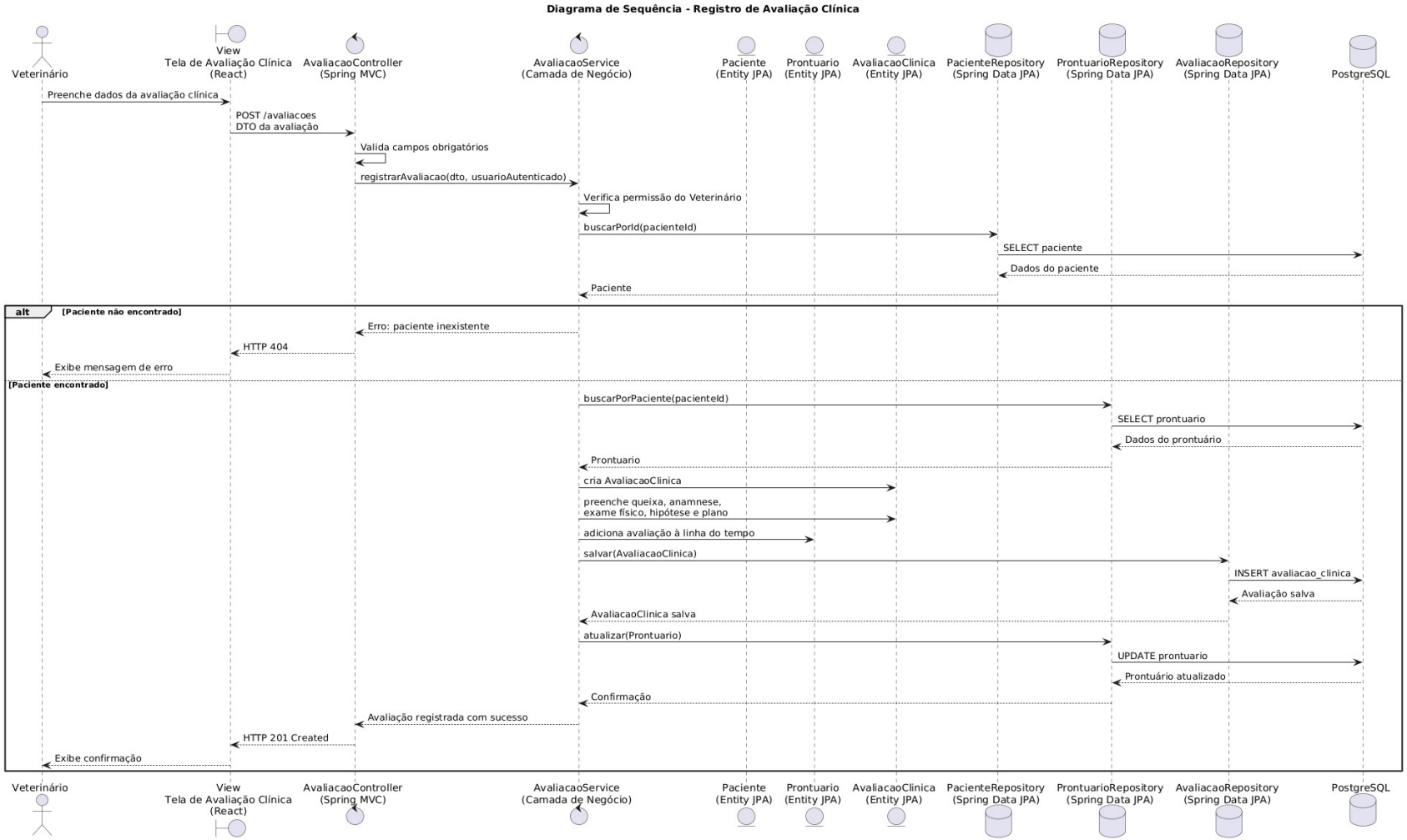


Figura 4 – Diagrama de Sequência do Registro de Avaliação Clínica

5. Visão de Implantação

A Visão de Implantação descreve como os componentes do sistema VetCare são distribuídos na infraestrutura responsável por sua execução. Essa visão complementa a Visão Lógica ao demonstrar em quais nós físicos ou ambientes os componentes da aplicação são implantados e como ocorre a comunicação entre eles.

Como o VetCare é uma aplicação web baseada em arquitetura cliente-servidor, sua implantação é composta por três elementos principais: o dispositivo do usuário, o servidor da aplicação e o servidor de banco de dados. O dispositivo do usuário executa a interface web por meio de um navegador moderno, permitindo o acesso de administradores, veterinários, tutores e apoio administrativo. O servidor da aplicação executa a API do VetCare desenvolvida em Java com Spring Boot, concentrando os controllers, serviços de negócio, mecanismos de segurança e acesso à persistência. O servidor de banco de dados executa o PostgreSQL, responsável pelo armazenamento persistente das informações clínicas e operacionais.

A comunicação entre o cliente e o servidor da aplicação ocorre por meio do protocolo HTTPS, utilizando chamadas REST para troca de dados entre front-end e back-end. A comunicação entre o servidor da aplicação e o banco de dados ocorre por meio de conexão JDBC, mediada pela camada de persistência com Spring Data JPA/Hibernate.

A arquitetura também prevê o uso de armazenamento externo para fotos, vídeos e documentos clínicos, como Amazon S3 ou MinIO. Essa decisão evita sobrecarregar o banco relacional com arquivos binários, mantendo no PostgreSQL apenas os metadados, vínculos e referências necessárias aos registros clínicos.

Essa configuração favorece segurança, manutenção, escalabilidade e disponibilidade, pois separa a interface, o processamento da aplicação, a persistência estruturada e o armazenamento de mídias.

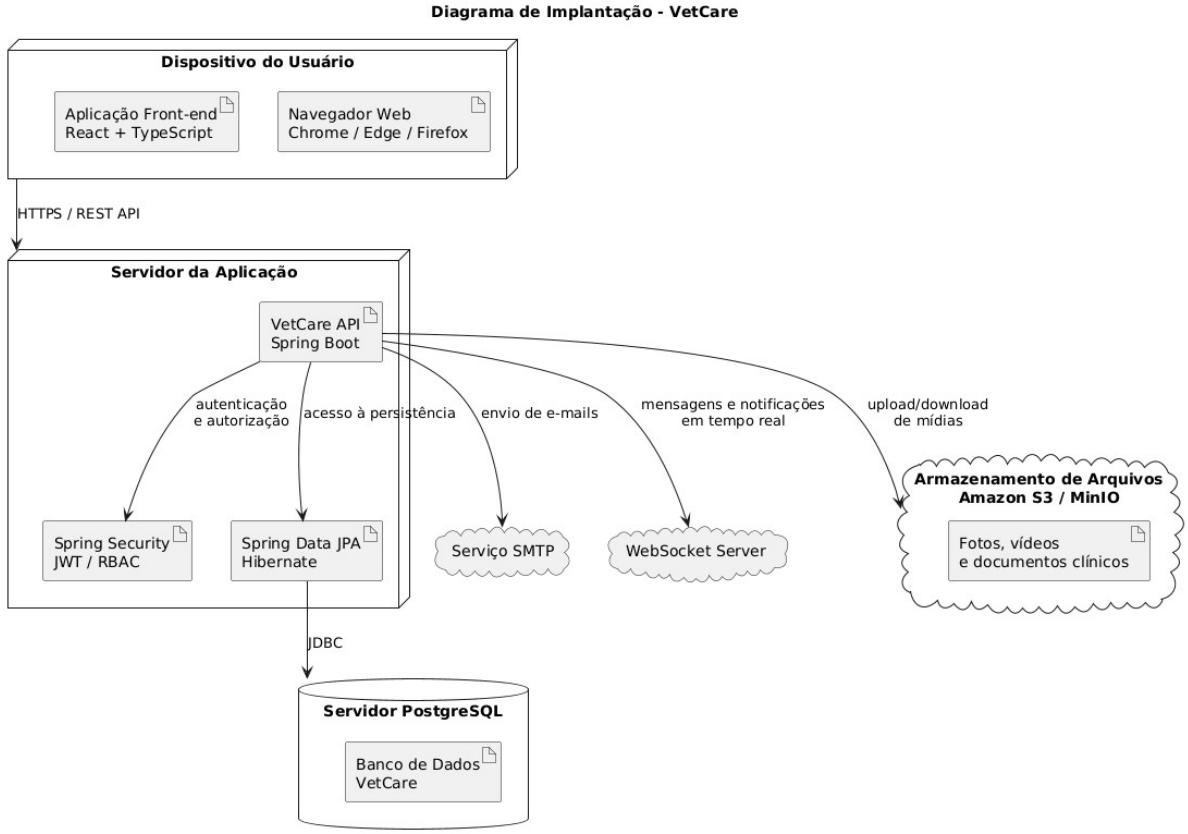


Figura 5 – Diagrama de Implantação do VetCare

6. Visão de Dados

A Visão de Dados descreve como as informações persistentes do VetCare são estruturadas, armazenadas e relacionadas. Essa visão complementa a Visão Lógica, pois transforma as principais classes e conceitos do domínio em estruturas de dados persistentes.

O VetCare utiliza banco de dados relacional PostgreSQL para armazenar dados estruturados, como usuários, perfis, tutores, pacientes, veterinários, agendas, sessões, avaliações clínicas, atendimentos fisioterapêuticos, prontuários, documentos, mensagens e notificações. A persistência desses dados é realizada pela aplicação back-end por meio de Spring Data JPA/Hibernate, permitindo o mapeamento entre as entidades Java e as tabelas do banco de dados.

No domínio do sistema, uma clínica possui usuários associados a diferentes perfis de acesso. O tutor é responsável por um ou mais pacientes/pets, enquanto o veterinário realiza avaliações, sessões e atendimentos fisioterapêuticos. O paciente possui prontuário, no qual são registrados de forma cronológica avaliações clínicas, atendimentos, documentos, mídias e demais informações relevantes ao acompanhamento do tratamento. As observações internas devem permanecer restritas aos perfis autorizados, garantindo a privacidade clínica definida nos requisitos do sistema.

As mídias, como fotos, vídeos e documentos clínicos, não devem ser armazenadas diretamente no banco de dados relacional. Esses arquivos são mantidos em armazenamento externo, como Amazon S3 ou MinIO, enquanto o banco armazena apenas os metadados, como nome do arquivo, tipo, caminho de acesso, data de envio, usuário responsável e vínculo com o prontuário, sessão ou atendimento correspondente.

Considerando a evolução futura do VetCare para uma solução SaaS/Multi-Tenant, o modelo de dados deve permitir a separação lógica das informações por clínica ou tenant. Dessa forma, os principais registros do sistema podem ser associados a uma clínica, garantindo isolamento lógico dos dados, controle de acesso adequado e possibilidade de expansão para diferentes organizações.

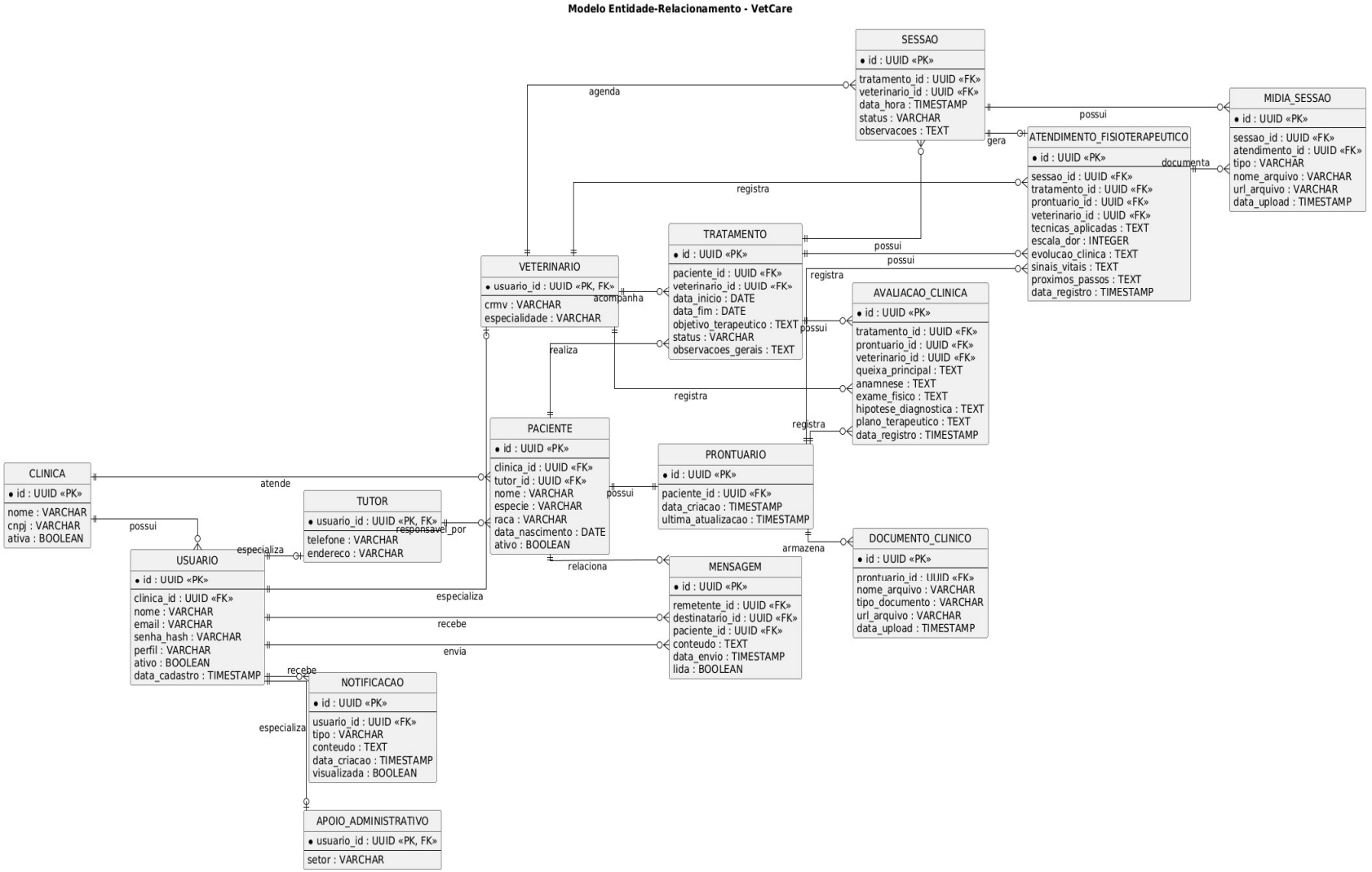


Figura 6 – Modelo Entidade-Relacionamento do VetCare

7. Volume e Desempenho

As estimativas de volume e desempenho foram definidas considerando a evolução do VetCare como uma plataforma web SaaS/Multi-Tenant, capaz de atender não apenas a clínica VetSPA, mas futuramente uma rede de clínicas veterinárias com diferentes usuários, pacientes, prontuários, documentos e mídias.

7.1 Estimativa de Usuários e Acessos

A estimativa média inicial considera até 1.000 usuários simultâneos, compostos por veterinários utilizando o sistema durante atendimentos, administradores e apoio administrativo gerenciando agenda e cadastros, além de tutores consultando prontuários, confirmando sessões e acompanhando o tratamento dos pets.

Em horários de maior demanda, como início de expediente, fechamento de atendimentos, envio de notificações ou confirmação de sessões, estima-se um pico de até 3.000 usuários simultâneos. A taxa estimada de requisições é de aproximadamente 150 requisições por segundo, com maior concentração em operações de agenda, autenticação, consulta de prontuário, envio de mensagens e notificações.

7.2 Dimensionamento de Dados

O volume inicial estimado é de 500 GB, considerando a possibilidade de migração de dados legados e o armazenamento de grande quantidade de mídias clínicas, como fotos e vídeos de evolução motora dos pacientes. O crescimento médio mensal estimado é de 20 GB, influenciado principalmente pelo anexo contínuo de mídias às sessões e pelo armazenamento de documentos clínicos.

Os logs de aplicação e auditoria devem ser mantidos por 6 meses, permitindo rastreabilidade de alterações, análise de falhas, auditoria de segurança e acompanhamento de eventos relevantes do sistema.

7.3 Metas de Desempenho e Disponibilidade

As consultas de agenda e prontuário devem apresentar tempo de resposta inferior a 2 segundos para 95% das requisições, conforme definido nos requisitos não funcionais. Para atender essa meta, a arquitetura prevê uso de paginação, índices em campos de busca, carregamento sob demanda de mídias e separação entre dados clínicos estruturados e arquivos externos.

A disponibilidade esperada do sistema é de 99,5% ao mês, excluídas janelas de manutenção programada. Para apoiar essa meta, a arquitetura prevê implantação em nuvem, monitoramento contínuo, health check, backup diário e mecanismos de recuperação.

O backup deve ser realizado diariamente, com garantia de zero perda de registros clínicos confirmados. Essa decisão está alinhada à necessidade de confiabilidade e integridade do prontuário clínico, evitando exclusão definitiva de registros e preservando o histórico de atendimento do paciente.

8. Referências

KRUTCHEN, Philippe. Architectural Blueprints: The “4+1” View Model of Software Architecture. IEEE Software, v. 12, n. 6, p. 42–50, 1995.

OBJECT MANAGEMENT GROUP. Unified Modeling Language Specification, version 2.5.1. OMG, 2017. Disponível em: https://www.omg.org/spec/UML/2.5.1/

SOMMERVILLE, Ian. Engenharia de Software. 10. ed. São Paulo: Pearson, 2019.

PRESSMAN, Roger S.; MAXIM, Bruce R. Engenharia de Software: uma abordagem profissional. 8. ed. Porto Alegre: AMGH, 2016.

FOWLER, Martin. UML Essencial: um breve guia para a linguagem-padrão de modelagem de objetos. 3. ed. Porto Alegre: Bookman, 2005.

VETCARE. Documento de Requisitos de Software. Versão 1.0, 2026.

VETCARE. Documento de Visão. Versão 1.0, 2026.